Projective Geometry as a Unified Framework for Exact Computational Geometry

Wai-Shing Luk

Abstract*—Classical geometry is usually taught as a disjoint collection of subjects: Euclidean, elliptic and hyperbolic geometry are presented as separate theories, and trigonometry is introduced through transcendental functions. This paper develops the opposite point of view. Starting from a single coordinate representation—homogeneous coordinates over an arbitrary field—we show how the projective plane, its incidence structure, the classical theorems of Pappus, Desargues and Pascal, conics and polarity, the Cayley-Klein family of metric geometries, and Wildberger's rational trigonometry all arise from one small set of operations: the dot product, the cross product, and parameterization along a line. The metric geometries differ only in the fundamental conic that defines the polarity operator; the measurements of quadrance and spread replace distance and angle without introducing square roots or trigonometric functions, and the five laws of rational trigonometry (Pythagoras, the triple-quad and triple-spread formulas, the cross law and the spread law) are stated over the same rational arithmetic. We then describe the design of the `projgeom` library, a four-layer trait and class hierarchy whose layers mirror the axiomatic hierarchy of the subject, implemented in Python, Rust and C++. Exact integer and rational arithmetic removes the robustness problems of floating-point predicates at the cost of an overflow and normalization budget; we report cross-language consistency verification (30/30 shared reference cases), a property-based test suite of 157 tests, and a microbenchmark study showing that the C++ and Rust cores execute a primitive operation in roughly 2.5–5 ns while the Python core, though 400–5000 times slower, remains below 150 microseconds per operation. We close with the limits of the approach: exactness that stops at the rationals, confinement to the projective plane, coordinate growth along long constructions, and the non-additivity of the rational measurements.

I. Introduction

Geometry is one of the oldest branches of mathematics, yet its modern presentation is fragmented. A student meets Euclidean geometry, then trigonometry with its sine and cosine, and only later—if at all—learns that Euclidean, elliptic and hyperbolic geometry are three members of a single projective family. The fragmentation is not intrinsic to the subject. As Arthur Cayley famously observed, projective geometry is all geometry: a single projective space, equipped with a single quadratic form, generates every classical metric geometry as a special case [1], [2]. What differs between geometries is not the incidence structure—which points lie on which lines—but the notion of perpendicularity, and that notion is encoded by one conic, the absolute or fundamental conic.

This unification has a computational counterpart that is less often emphasized. If one accepts homogeneous coordinates over an arbitrary field, then points and lines become indistinguishable three-vectors, incidence becomes a dot product, and the join of two points and the meet of two lines become the same cross product. Duality is therefore not a theorem to be proved but a symmetry of the representation. The measurement operations of the metric geometries—the polar of a point, the altitude of a triangle, the reflection in a line, the midpoint of two points, the angle bisector of two lines—are all expressible through the same dot and cross products applied to a small matrix that represents the fundamental conic. Finally, Wildberger's rational trigonometry [3] observes that the natural measurements in this framework are the *quadrance* (a squared distance) and the *spread* (a squared ratio of lengths), both rational functions of the coordinates. Distance and angle are then recovered only when needed, by taking a square root or an inverse sine, and the exactness of the underlying arithmetic is preserved for as long as possible.

This paper is organized around that unifying principle and around the design decision it forces on any implementation. The verification of thousands of geometric predicates is safer, and often faster, when it is performed in exact arithmetic rather than in floating point; the classic robustness results on geometric predicates make this concrete [4], [5], [6]. But exact arithmetic over the rationals is not free: every operation carries an overflow check and a greatest-common-divisor normalization, and the choice of number system propagates into every routine. The contribution of this paper is therefore twofold. First, it presents a self-contained and uniform account of the projective-to-Cayley-Klein-to-rational-trigonometry chain, with all constructions expressed through a minimal set of operations that an implementation can be built on directly. Second, it reports an implemented system, the `projgeom` library, whose design is deliberately shaped by the mathematical hierarchy, and it measures the consequences of that design.

Specifically, our contributions are the following.

1. We give a coordinate-first treatment of the projective plane in which incidence, duality, join and meet are consequences of the dot and cross products, and we show that the same formulas hold over the integers, the rationals, finite fields and the complex numbers (Section III).
2. We recast the classical theorems—Pappus, Desargues,

Pascal, and the theory of quadrangular sets and harmonic conjugates—as identities that a symbolic or exact computation can certify, and we describe how these checks are implemented (Section IV).

3. We develop conics and polarity, and from the fundamental conic we derive the Cayley-Klein classification together with the constructions of perpendicularity, altitude, orthocenter, reflection, midpoint and angle bisector (Section V, Section VI).
4. We state the five laws of rational trigonometry in their universal form, relate them to the Archimedes function, Heron's and Brahmagupta's formulas, and the rational form of Ptolemy's theorem, and we treat the projective line and its connection to the Lorentz transformations of special relativity (Section VII, Section VIII).
5. We describe a four-layer software architecture in which each layer corresponds to one level of the axiomatic hierarchy, and we report cross-language consistency verification and microbenchmarks across Python, Rust and C++ (Section IX, Section IX-D).
6. We analyze the cost and the pitfalls of exact arithmetic—overflow guards, redundant gcd normalizations, and a latent division-by-zero failure—and we quantify the payoff of repairing them (Section IX-D).

II. Related Work

Projective geometry has a long and settled literature. Coxeter's *Projective Geometry* [1] remains the canonical synthetic and analytic treatment, and his *Introduction to Geometry* [7] connects the projective viewpoint to the classical metric theorems. The axiomatic foundations are laid out by Veblen and Young [8] and, in a compact analytic form, by Samuel [9]; Pedoe [10] and Hartshorne [11] provide the bridge from Euclid, and Richter-Gebert [12] combines rigorous theory with computational illustration. Stillwell [13] and Klein's Erlangen program [2] supply the group-theoretic reading of the unification, and Yaglom [14] links the hyperbolic plane to the geometry of the projective line.

Rational trigonometry in its modern form is due to Wildberger [3]; the hyperbolic extension and the universal laws are developed in the same monograph and its sequels [15]. Our treatment follows his notation, with quadrance and spread as the primary measurements.

The question of exactness in geometric computation is treated by Yap [4] and Shewchuk [5], and the failure modes of naive floating-point predicates are catalogued by Kettner et al. [6]. Our library occupies a different point in the design space: rather than repairing floating-point predicates with adaptive precision, it fixes the number system once, at the top, and carries exact integers or rationals throughout the core.

Finally, the presentation here is distilled from a family of open-source implementations of the same theory—`projgeom-py` [16], `projgeom-rs` [17] and `projgeom-cpp` [18]—together with supporting exact-rational libraries [19], [20]. The cross-language and benchmark results of Section IX-D are drawn from that family.

III. The Projective Plane

A. Homogeneous Coordinates

We work with two primitive sorts of object. A *point* is written as $p = [x, y, z]$ and a *line* as $l = [a, b, c]$, but both are simply nonzero elements of a three-dimensional vector space over a field K. Two vectors that differ by a nonzero scalar represent the same object, $[x, y, z] = \lambda[x, y, z]$ for every $\lambda \neq 0$, so what a point really denotes is a one-dimensional subspace, or a *projective point* of the plane $\mathbb{P}^2(K)$. The condition that a triple be nonzero is the only degeneracy that is excluded: $[0, 0, 0]$ is neither a point nor a line.

The link between algebra and geometry is the incidence relation. A point p lies on a line l precisely when

$$p \cdot l = xa + yb + zc = 0. \tag{1}$$

This single equation already exhibits the duality of the plane. If we read p as a point and l as a line, Eq. 1 says that the point lies on the line; but the expression is symmetric in p and l, so the same statement can be read as a point l lying on a line p. The incidence relation does not distinguish the two sorts, and neither do the operations that we build on it.

Concretely, homogeneous coordinates encode the ordinary affine plane by fixing a line at infinity. In the *Euclidean* normalization we take $l_\infty = [0, 0, 1]$, treat the affine point (x, y) as the projective point $[x, y, 1]$, and interpret a point $[x, y, 0]$ as the direction of the vector (x, y)—a *point at infinity*. A line $[a, b, c]$ then satisfies $ax/z + by/z + c = 0$ for its finite points, and constant triples are identified, so that $[1, 5, 6]$ and $[-10, -50, -60]$ both represent $(1/6, 5/6)$. Fig. 1 shows the two sorts and their incidence.



Figure 1: Incidence of points and lines in the projective plane.

B. Join, Meet and Duality

Two distinct points determine a line and two distinct lines determine a point. In homogeneous coordinates both operations are the cross product: the *join* of points p and q is the line

$$[p, q] = p \times q = [\, y_p z_q - z_p y_q,\ z_p x_q - x_p z_q,\ x_p y_q - y_p x_q \,], \tag{2}$$

and the *meet* of lines l and m is the point

$$[l, m] = l \times m, \tag{3}$$

with the same component formula. That $p \times q$ is incident with both p and q follows from the scalar triple product

$p \cdot (p \times q) = 0$; the join is unique, up to the identification of proportional triples, because the cross product of two independent vectors is a one-dimensional space. The map that takes a point to a line—and is therefore used to pass between the two descriptions—is written *aux* in the implementation and is a bijection on valid objects.

The formal statement of duality is that the plane is symmetric under the interchange of the words "point" and "line," provided join and meet are interchanged as well. Every construction and every theorem therefore has a *dual* obtained by this substitution. The two operations share a single implementation; only the type of the result changes. Fig. 2 illustrates the symmetry: the join of two points and the meet of two lines are the same computation viewed from the two sides.

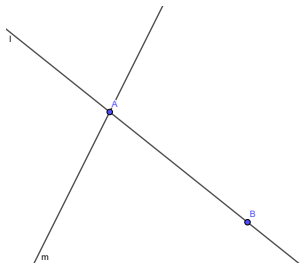


Figure 2: Duality of join and meet: the cross product computes both.

Two objects of the same sort are equal precisely when their cross product vanishes, $p = q \iff p \times q = 0$, and this is the equality test adopted by the library. The dot product, by contrast, is the *basic metric* between a point and a line: it may be positive, negative or zero, and it vanishes exactly when the two are incident. It is important that no orientation or metric is required for any of this; the plane needs no notion of distance to be a projective plane.

C. Points on a Line and the Number System

Given two distinct points p and q, every point on the line $[p, q]$ is a *linear combination*

$$\mathrm{plckr}(\lambda, p, \mu, q) = \lambda p + \mu q, \tag{4}$$

with λ, μ not both zero. This parameterization is the workhorse of the whole development: midpoints, harmonic conjugates, points on a side of a triangle and intersections are all computed by choosing the scalars λ, μ correctly. When the coordinates are integers, the representatives may be chosen so that the combination stays integral, which is why the parameterization returns exact objects rather than rounded ones. The two coefficients are conventionally named λ and μ, and the operation is exposed as `parametrize` (or `plckr`) in the implementations.

The choice of field K is deliberately unconstrained. The same formulas hold over the integers $\mathbb{Z}$, the rationals $\mathbb{Q}$, the reals $\mathbb{R}$, the complex numbers $\mathbb{C}$, and the finite fields $\mathrm{GF}(p)$ for prime p (and their prime-power extensions). Multiplicative inverses are never required for incidence, join or meet, so a ring rather than a field would suffice for the incidence layer; inverses appear only when a metric is introduced. This is why the library can work with integers throughout most of its operations and reserve rationals or floating point for the measurements that genuinely need them. Even coordinates are not strictly necessary—the "poker card" example of the accompanying notes realizes a finite projective plane of order three on 13 symbols—but every construction in this paper is stated for homogeneous coordinates, and we restrict attention to that case.

The choice of line at infinity is itself part of the projective data. Any line may be declared the line at infinity, and the affine picture that results is a perspective view of the plane (Fig. 3). The same incidence structure then yields a different-looking Euclidean drawing, which is the geometric content of the remark that the projective plane privileges no line. This flexibility is what allows one implementation to cover Euclidean, elliptic and hyperbolic geometry without changing the underlying incidence code.

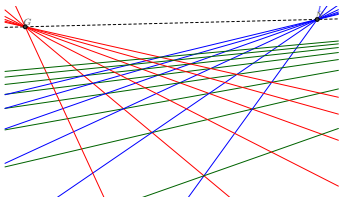


Figure 3: The perspective view of the Euclidean plane obtained by choosing a different line at infinity.

IV. Projective Configurations and Theorems

A. Collinearity, Concurrence and Triangles

Three points are *collinear* when they lie on a common line, and three lines are *concurrent* when they pass through a common point; the two notions are dual. In coordinates, A, B, C are collinear exactly when $AB \cdot C = 0$, where $AB = A \times B$; the conjunction of these conditions for a longer list of points is written coI in the source. A non-collinear triple of points is a *triangle* $\{ABC\}$, and a non-concurrent triple of lines is a *trilateral* $\{abc\}$; the triangle is dual to the trilateral whose vertices are bc, ac and ab (Fig. 4).

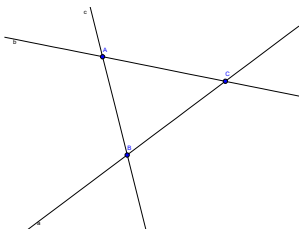


Figure 4: A triangle and its dual trilateral: the vertices of one are the sides of the other.

B. Pappus and Desargues

Two theorems anchor the development, one older than the axiomatic method and one that characterizes the plane.

Theorem 1 (Pappus). Let A, B, C be distinct collinear points and let D, E, F be distinct collinear points on another line. Then the three meets $X = AE \cap BD$, $Y = AF \cap CD$ and $Z = BF \cap CE$ are collinear.

Theorem 2 (Desargues). Two triangles are *perspective from a point* if the lines joining corresponding vertices are concurrent; they are *perspective from a line* if the intersections of corresponding sides are collinear. The two conditions are equivalent. The theorem is self-dual.

Both theorems are verified without any metric. For Pappus, one parameterizes $C = \lambda_1 A + \mu_1 B$ and $F = \lambda_2 D + \mu_2 E$, expands the three meets as functions of the six base points and the four scalars, and observes that $X \cdot (Y \times Z)$ simplifies to zero; the identity can be checked by a computer algebra system, and it holds for *any* underlying field, not merely over the reals. Desargues is handled the same way, with the self-duality supplying the converse. Fig. 5 and Fig. 6 show representative configurations.

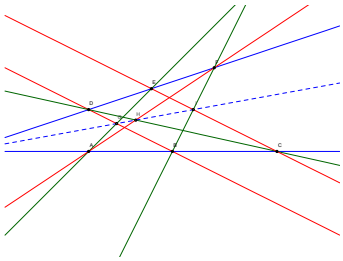


Figure 5: An instance of Pappus's theorem: the three cross-meets are collinear.

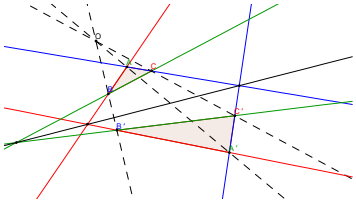


Figure 6: An instance of Desargues's theorem: perspective from a point and from a line coincide.

The same mechanism checks Pappus and Desargues within a finite geometry: the poker-card plane of order three satisfies Pappus, which is one way to see that the theorem is not a statement about continuity or the real numbers.

C. Quadrangular Sets and Harmonic Conjugates

Let P, Q, R, S be four points in general position—a *quadrangle*, so that no three are collinear. The six joins of the quadrangle meet a further line l in six points, which, when grouped as pairs,

$$\begin{aligned} A &= PQ \cap l, \quad D = RS \cap l, \\ B &= QR \cap l, \quad E = PS \cap l, \\ C &= PR \cap l, \quad F = QS \cap l, \end{aligned} \tag{5}$$

form a *quadrilateral set* $(AD)(BE)(CF)$. Fig. 7 shows the construction. A quadrilateral set in which two of the pairs coincide, $A = D$ and $B = E$, is a *harmonic set* $H(AB, CF)$; the points C and F are then *harmonic conjugates* with respect to A and B. The harmonic relation is preserved by projectivities and, in particular, by perspectivities—the two statements are the projective and the perspective half of the classical result.

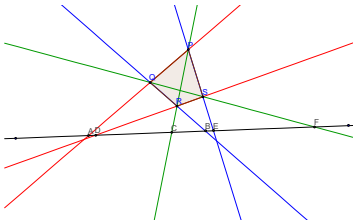


Figure 7: A quadrilateral set: the six meets of a complete quadrangle with a line.

D. Cross Ratio

The fundamental invariant of the projective group is the cross ratio. For four collinear points A, B, C, D, choose any point O not on their line; then

$$R(A, B; C, D) = \frac{(OA \cdot C)(OB \cdot D)}{(OA \cdot D)(OB \cdot C)}, \tag{6}$$

and this value is independent of O. In the homogeneous coordinates of the line the same quantity is the ratio of two determinants,

$$R(A, B; C, D) = \frac{[AC]\,[BD]}{[AD]\,[BC]}, \tag{7}$$

where $[AC]$ denotes the two-by-two determinant of the chosen coordinate representatives. The cross ratio is real for four collinear real points and equals -1 exactly for a harmonic range. Any nonsingular linear map of the vector space—the coordinate form of a *projective transformation*—scales all four determinants by the same factor and therefore leaves R invariant.

A projective transformation is thus any invertible 3×3 matrix acting on the coordinate vectors,

$$p' = Tp, \qquad l' = (T^{-1})^{\mathsf{T}} l, \tag{8}$$

the dual action on the line being forced by the requirement that incidence be preserved. Projectivities and perspectivities are ordered-set versions of the same idea applied to collinear or concurrent configurations, and they compose in the expected way, with the caveat that perspectivity is not transitive in general (Fig. 8).

V. Conics and Polarity

A *polarity* is a projective correlation of period two: a bijection taking points to lines and back, whose square is the identity. Writing the polar of a point A as $A^{\perp}$ and the pole of a line a as $a^{\perp}$, a polarity satisfies $(A^{\perp})^{\perp} = A$ and $(a^{\perp})^{\perp} = a$ except in degenerate cases. A point that lies on its own polar is *self-conjugate*, and the locus of self-conjugate points is a *conic*. The polars of the points

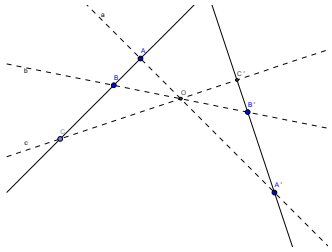


Figure 8: A perspectivity from a center maps one range of collinear points onto another.

of a conic are its tangents; any other line is a *secant* or a *nonsecant* according to whether it meets the conic twice or not at all.

Polarity is more general than the conic that it defines, because a polarity may have no real self-conjugate points—its conic may be entirely complex. The construction of the polar of a point C not on the conic is elementary: draw two secants PQ and RS through C; the polar is the line joining the meets $QR \cap PS$ and $RP \cap QS$. Dually, the pole of a secant is the meet of the polars of any two of its points, and the tangent at a point of the conic is the join of that point with the pole of any secant through it (Fig. 9). These constructions require no metric and are the reason the Cayley-Klein program of the next section is available at all: a metric geometry is a projective plane together with a chosen polarity.

Pascal's theorem belongs here as the conic counterpart of Pappus: if a hexagon is inscribed in a conic, the three pairs of opposite sides meet in collinear points. It reduces to Pappus in the degenerate case in which the conic splits into two lines, which is the classical way in which the two theorems are related (Fig. 10).

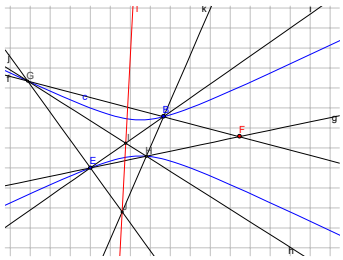


Figure 9: Constructing the polar of a point from two secants through it.

VI. Cayley-Klein Geometry

A. The Fundamental Cone

The Cayley-Klein construction restricts a projective plane by a fixed polarity, represented by a *fundamental cone* $(\mathbf{A}, \mathbf{B})$. The two matrices act on points and lines respectively,

$$[A^{\perp}] = \mathbf{A}[A], \qquad [a^{\perp}] = \mathbf{B}[a], \tag{9}$$

so that the pole and the polar are computed by a single matrix–vector product. The projectivization of the cone

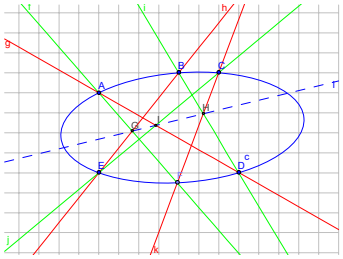


Figure 10: An instance of Pascal's theorem: the opposite-side meets of an inscribed hexagon are collinear.

Table I: The Cayley-Klein family. Only the fundamental conic, hence the action of $\perp$, differs between geometries.

Geometry	Fundamental conic	$p^{\perp}$
Elliptic	$x^2 + y^2 + z^2 = 0$	(x, y, z)
Hyperbolic	$x^2 + y^2 - z^2 = 0$	$(x, y, -z)$
Euclidean	$z = 0$ (degenerate)	$[0, 0, 1]$
Pseudo-Euclid	$z = 0$, sign $-$ (degenerate)	$(x, -y, 0)$

is the absolute figure of the geometry; in the hyperbolic model it is the unit circle, sometimes called the *null circle*, and the Euclidean plane is recovered as the limiting case in which the absolute is the line at infinity. Fig. 11 shows a fundamental cone together with a pole–polar pair.

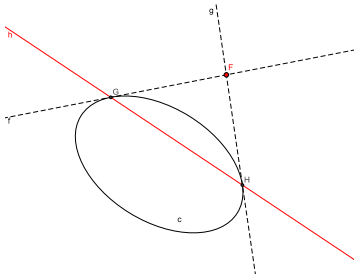


Figure 11: A fundamental cone with a pole and its polar defining a Cayley-Klein metric.

B. Classification

The sign pattern of the quadratic form $x \cdot x^{\perp} = [x]^{\mathsf{T}}\mathbf{A}[x]$ classifies the metric geometry. Table~I lists the four cases that matter: elliptic geometry, whose absolute is imaginary and whose polarity is self-dual; hyperbolic geometry, with the real absolute $x^2 + y^2 - z^2 = 0$; Euclidean geometry, the degenerate limit; and pseudo-Euclidean geometry, the remaining degenerate case. Only the action of $\perp$ distinguishes them; the incidence structure is common to all.

C. Perpendicularity, Altitude and Orthocenter

With a polarity in hand, perpendicularity is immediate. A line l is *perpendicular* to a line m when $l^{\perp}$ is incident with m, that is, when $m^{\mathsf{T}}\mathbf{B}l = 0$; dually, a point p is perpendicular to a point q when $q^{\mathsf{T}}\mathbf{A}p = 0$. The polar of a line, through a given point, is its *altitude*: the altitude of the line l through the point p is the join

$[p, l^\perp]$. Consequently the three altitudes of a non-degenerate triangle are concurrent, and their common point is the *orthocenter*. Unlike the Euclidean case, the orthocenter of a triangle in a general Cayley-Klein geometry need not lie inside the triangle, and the operation is available in elliptic and hyperbolic geometry as well. Fig. 12 gives an instance. A second theorem of interest is that the orthocenter relation is involutive on the triangle: if O is the orthocenter of $\{ABC\}$, then A is the orthocenter of $\{OBC\}$ (Fig. 13).

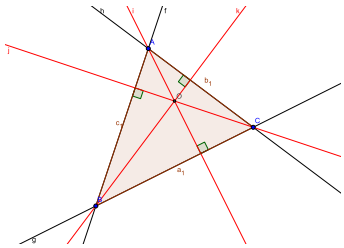


Figure 12: The orthocenter: the three altitudes of a triangle are concurrent.

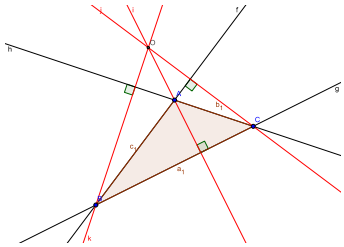


Figure 13: The orthocenter relation is involutive: A is the orthocenter of the triangle formed by O, B and C.

D. Involution, Reflection, Midpoint and Bisector

An *involution* is a projective transformation that is its own inverse, $\tau(\tau(p)) = p$. Every involution is determined by a *mirror* line m and a *center* point o: the points of m are fixed, and the lines through o are fixed as lines. Its point-transformation matrix is

$$T = (o^\mathsf{T} m)\,\mathrm{I} - 2\,o\,m^\mathsf{T}, \tag{10}$$

so that

$$\tau(p) = (o \cdot m)\,p - 2\,(m \cdot p)\,o. \tag{11}$$

When the center is the pole of the mirror, $o = m^\perp$, the fundamental cone is preserved by τ. In Euclidean geometry $m^\perp$ is the point at infinity in the direction perpendicular to m, and τ is the ordinary reflection in the line m; in elliptic geometry $m^\perp$ is finite and τ is a rotary reflection. This is the cleanest illustration of the Cayley-Klein thesis: one formula, two familiar transformations, the difference being only the value of the polarity. Reflection is exposed in the library as an involution whose center is $m^\perp$.

Two further constructions are dual to each other. The *midpoints* of two points p and q are

$$r = \sqrt{\Phi(p)}\,q \pm \sqrt{\Phi(q)}\,p, \qquad \Phi(p) = p^\mathsf{T}\mathbf{A}p, \tag{12}$$

and the *angle bisectors* of two lines l and m are

$$b = \sqrt{\Phi(m)}\,l \pm \sqrt{\Phi(l)}\,m. \tag{13}$$

In Euclidean geometry one of the two midpoints is a point at infinity and the other is the familiar midpoint; the two bisectors are perpendicular. In general the midpoint may be irrational, and it may even be complex although both inputs are real. Fig. 14 shows the construction of a pair of angle bisectors through the fundamental conic. The midpoint theorem and the complete angle-bisector theorem state that the three midpoints of a triangle lie on a line and that the corresponding bisector points are collinear; both are instances of the same duality (Fig. 15).

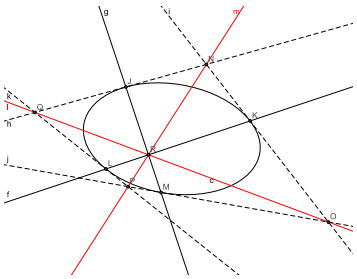


Figure 14: Constructing a pair of angle bisectors from the fundamental conic.

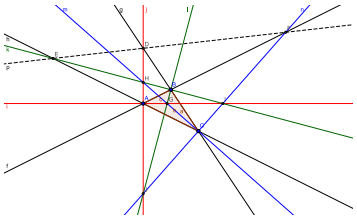


Figure 15: An instance of the complete angle-bisector theorem: one set of bisectors is concurrent and the corresponding meets are collinear.

VII. Rational Trigonometry

The measure of a Cayley-Klein geometry is most cleanly expressed without roots. Define the *Omega function* of an object by

$$\Omega(x) = x \cdot x^\perp = [x]^\mathsf{T}\mathbf{A}[x], \tag{14}$$

so that $\Omega(A) = A^\mathsf{T}\mathbf{A}A$ for a point and $\Omega(a) = a^\mathsf{T}\mathbf{B}a$ for a line. The *quadrance* between points A and B and the *spread* between lines l and m are

$$q(A,B) = \frac{\Omega(AB)}{\Omega(A)\,\Omega(B)}, \qquad s(l,m) = \frac{\Omega(lm)}{\Omega(l)\,\Omega(m)}, \tag{15}$$

where AB is the join and lm is the meet. Quadrance is the squared distance in the sense that $q(A,B) = d(A,B)^2$ in Euclidean geometry, while spread is the squared sine of the angle; in hyperbolic geometry the identification is $q = \sinh^2 d$ and $s = \sin^2\theta$, and in elliptic geometry it is $q = \sin^2 d$. All three cases are captured by the single formula Eq. 15, and it is invariant under every projective transformation that preserves the fundamental cone. This

is Wildberger's rational trigonometry [3], and its virtue for computation is that Eq. 15 contains no square root and no transcendental function.

For triangles the measurements satisfy a short list of polynomial identities. Let q_1, q_2, q_3 be the quadrances between the three pairs of vertices and s_1, s_2, s_3 the spreads between the three pairs of sides of the dual trilateral. Then, universally,

- (Triple quad formula) if the three points are collinear,
$$(q_1 + q_2 + q_3)^2 = 2(q_1^2 + q_2^2 + q_3^2) + 4q_1q_2q_3; \quad (16)$$
- (Triple spread formula) if the three lines are concurrent,
$$(s_1 + s_2 + s_3)^2 = 2(s_1^2 + s_2^2 + s_3^2) + 4s_1s_2s_3; \quad (17)$$
- (Spread law) the three ratios agree,
$$q_1/s_1 = q_2/s_2 = q_3/s_3; \quad (18)$$
- (Cross law) the three quadrances and the spread satisfy
$$\big(s_1s_2q_3 - (s_1 + s_2 + s_3) + 2\big)^2 = 4(1 - s_1)(1 - s_2)(1 - s_3). \quad (19)$$
- (Pythagoras) if $L_1 \perp L_2$, so that $s_3 = 1$, then
$$q_3 = q_1 + q_2 - q_1q_2. \quad (20)$$

In Euclidean geometry the last term of the triple quad formula drops out and Pythagoras reads $q_3 = q_1 + q_2$; the Euclidean formulas are recovered as the degenerate case of the universal ones. The spread law is the rational analogue of the sine law, with quadrance in place of length, and the cross law is the analogue of the cosine law; the right-parallax theorem, which reads $q = (s-1)/s$, or equivalently $s = 1/(1-q)$, is the analogue of the right-triangle relation for a triangle with one spread equal to zero. One practical consequence, visible in the notes and in the implementation, is that a small dispersion-measuring loop on points of the sphere can compare spreads directly instead of comparing arc lengths, and the two agree.

The polynomial identities extend to polygons through the *Archimedes function*

$$\begin{aligned}\mathrm{Ar}(q_1, q_2, q_3) &= 4q_1q_2 - (q_1 + q_2 - q_3)^2 \\ &= (q_1 + q_2 + q_3)^2 - 2(q_1^2 + q_2^2 + q_3^2),\end{aligned} \quad (21)$$

which vanishes exactly when the three quadrances are collinear and satisfies $16 \cdot \text{area}^2 = \mathrm{Ar}(q_1, q_2, q_3)$ for a triangle. Heron's formula is its classical specialization to side lengths (Fig. 17), and the *quadruple quad formula*

$$\begin{aligned}Q(a, b, c, d) = \big[(a + b + c + d)^2 - 2(a^2 + b^2 + c^2 + d^2)\big]^2 \\ - 64\, abcd\end{aligned} \quad (22)$$

serves the same role for quadrilaterals. Brahmagupta's formula gives the area of a cyclic convex quadrilateral as the square root of a product of linear factors, and the cyclic-quadrilateral theorem turns this into a quadratic equation in the area. Most elegantly, the rational form of Ptolemy's theorem states that a quadrilateral with quadrances Q_{ij} is cyclic if and only if

$$\mathrm{Ar}(Q_{12}Q_{34},\, Q_{23}Q_{14},\, Q_{13}Q_{24}) = 0. \quad (23)$$

Because the three arguments enter symmetrically, the rational form does not require convexity, unlike the classical product-of-diagonals statement. Fig. 16 illustrates the cyclic quadrilateral. The proof is a one-line consequence of the unit-circle parameterization $uc = [\lambda^2 - \mu^2,\, 2\lambda\mu,\, \lambda^2 + \mu^2]$: substituting the four parameterizations and simplifying the expression in Eq. 23 yields the zero polynomial identically, which an external symbolic algebra system confirms; the library itself performs no symbolic computation.

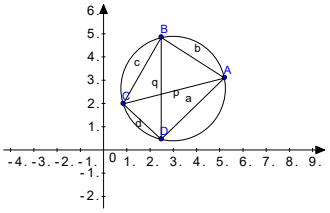


Figure 16: Ptolemy's theorem: the rational form holds without a convexity hypothesis.

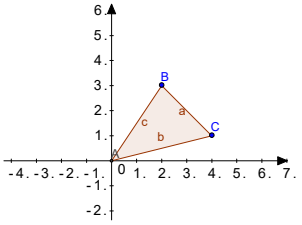


Figure 17: Heron's formula: the area of a triangle from its side lengths, the Euclidean case of the Archimedes function.

VIII. The Projective Line

The one-dimensional case is worth isolating, because it is where the projective line meets physics. A point of $\mathbb{P}^1$ is a nonzero pair $[x, y]$, and the *cross ratio* of four points is

$$R_1(a, b; c, d) = \frac{(a \times c)(b \times d)}{(a \times d)(b \times c)}, \quad (24)$$

the one-dimensional instance of Eq. 7. The most general invertible transformation is the *Mobius transformation*

$$z' = \frac{az + b}{cz + d}, \qquad z = x/y, \quad (25)$$

which has two fixed points on the projective line when the fixed points are counted over the algebraic closure and infinity is allowed as a value. This is the coordinate form of a projective transformation of the line, and it is the appropriate setting for the Minkowski geometry of special relativity. A velocity is the point $[x, t]$ with speed $v = x/t$, and the *relative speed* of two such points is

$$\mathrm{Speed}(p, q) = \frac{(x_pt_q - t_px_q)^2}{(x_p^2 - t_p^2)(x_q^2 - t_q^2)} = \frac{(v_p - v_q)^2}{(v_p^2 - 1)(v_q^2 - 1)}, \quad (26)$$

which is precisely the same rational expression as Eq. 15, specialized to the projective line. When the speed of light is normalized to one, the relative speed never exceeds one provided each of v_p, v_q lies in the unit interval; the additivity of velocities is replaced by the projective composition of Mobius transformations. The 1D theory is thus not a toy but the exact algebraic skeleton of Lorentz geometry (Fig. 18).

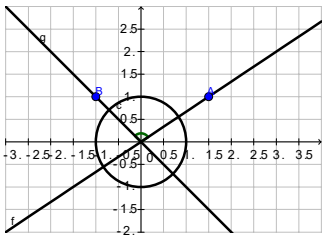


Figure 18: The projective line in its elliptic model; its measurement is the same rational expression as the quadrance.

IX. The Software Architecture

A. Design Goals

The library is organized around four goals. First, *exactness*: the core should use exact integers or rationals wherever possible, so that incidence and equality predicates are decided exactly rather than within a tolerance. Second, *uniformity*: the same routine should serve a point and a line, a Euclidean and a hyperbolic configuration, by abstracting over the polarity. Third, *layering*: the code should expose exactly the operations that the mathematics of each level requires, so that an impossible operation—a perpendicular in a plane with no metric, say—is not merely discouraged but unrepresentable. Fourth, *portability*: the same algorithms should be implementable, with the same names and the same results, in more than one language.

B. Four Layers

The implementation realizes the first three goals with a four-layer hierarchy whose layers correspond to the axiomatic levels of the subject. Table II lists them. The first layer provides incidence: two points meet to a line, two lines meet to a point, and a point is incident with a line. The second layer adds a bilinear form, the auxiliary construction of a non-incident object, and the parameterization of a line. The third layer adds the single operation that defines a metric geometry, the polarity *perp*. The fourth layer combines the second and third and is the home of the measurements, quadrance and spread. Each layer is a strict extension of the preceding one.

The layering is not decoration. A pure projective object implements the first two layers and no more, and the type system therefore rejects an attempt to compute a perpendicular of it at compile time (in Rust and C++) or at the level of the class hierarchy (in Python). A Euclidean, elliptic or hyperbolic object implements all four. The only substantive difference between the three metric geometries

Table II: The four-layer architecture. Each layer corresponds to one level of the axiomatic hierarchy and adds only the operations that level needs.

Layer	Trait	Operations
1	`ProjectivePlanePrimitive`	meet, incident
2	`ProjectivePlane`	aux, dot, parametrize
3	`CayleyKleinPlanePrimitive`	perp
4	`CayleyKleinPlane`	quadrance, spread

is the three-line definition of *perp*: elliptic geometry returns the coordinates unchanged, hyperbolic geometry negates the last coordinate, and Euclidean geometry maps every point to the line at infinity and every line to its direction vector. Everything above that definition is shared.

C. Core Algorithms

The shared primitives are the dot product, the cross product, and the parameterization Eq. 4. From them the harmonic conjugate of three collinear points a, b, c is

$$\begin{aligned} harm_conj(a,b,c) &= a.parametrize(\, lc \cdot b,\ b,\ lc \cdot a\,), \\ lc &= (a \times b)^{\perp} \times c, \end{aligned} \tag{27}$$

where $(a \times b)^{\perp}$ is the auxiliary line through c. The orthocenter is the meet of two altitudes, each altitude an instance of the join of a point with a polar; the reflection is an involution centered at a polar; and the measurements are computed from Eq. 15. The reuse is the point: once the polarity is fixed, the metric toolkit is a short composition of the same two operations.

The arithmetic that backs these routines deserves a note. Integers and rationals are the default; a rational is kept in lowest terms with a positive denominator, and every operation performs a greatest-common-divisor normalization. Floating point appears only at the boundary, when a distance or an angle is actually requested. This is the design that makes the cross-language consistency of Section IX-D meaningful: because the algorithms are polynomial identities over the rationals, different languages must produce numerically identical results, not merely close ones.

D. Cross-Language Verification and Benchmarks

The three implementations expose the same function names and are checked against a common battery of reference cases drawn from the doc-test suites of the Rust and C++ cores. Thirty reference cases—covering vector operations, projective point and line operations, the harmonic conjugate, the three polarities, eight Euclidean routines, the cross ratio, the conic and transform modules—are evaluated identically in all three languages, and all thirty pass. The Python core carries a further property-based suite of 157 collected tests, 91 of them Hypothesis tests that assert algebraic invariants—the idempotence of duality, the involution property of reflection, collinearity constraints under construction—across the Euclidean, Cayley-Klein, elliptic,

hyperbolic and perspective models, in addition to source-level doctests. The performance evidence is deliberately atomic: Section IX-D reports five primitive operations, and the single benchmark module of the repository measures twenty-one such operations in isolation. We do not report workload-level benchmarks (convex hulls, Voronoi diagrams, meshes) and no comparison is made against adaptive-precision predicate libraries such as Shewchuk's [5] or against CGAL, which we cite as background rather than as baselines; the contribution here is a design and a reference implementation, not a performance claim. The verification does surface an error of the kind that exactness makes visible: the Rust documentation of the Archimedes function asserted that $\mathrm{Ar}(1,2,3) = 0$, whereas the formula gives $4 \cdot 1 \cdot 2 - (1 + 2 - 3)^2 = 8$; the assertion was wrong, and a correct zero case, $\mathrm{Ar}(1,4,9) = 16 - 16 = 0$, replaced it. Table III reports microbenchmarks of representative operations.

The C++ core executes a primitive operation in roughly two and a half nanoseconds, and Rust is within a factor of two of C++; both benefit from inlining and from the fact that the core operations are a handful of multiplies and adds. Python is between four hundred and five thousand times slower, an unsurprising figure for an interpreted language with arbitrary precision integers, but even the most expensive routine measured—the inverse of a 3×3 transform, at about 149 microseconds—is well within interactive budgets. The practical guidance is the familiar one: millions of operations per second call for C++ or Rust; interactive geometry, visualization and teaching are served well by Python.

Exact arithmetic has a measurable cost that is worth isolating. In the rational library, every arithmetic operation on in-range values paid for a division-based overflow guard and for a redundant normalization. An instrumented run of two million operations triggered the overflow fallback zero times, so the guard was pure overhead, and neutralizing it recovered about twenty percent of the runtime on small rationals and about ten percent on large ones. The same guard contained a latent crash: the check $\mathrm{MAX}/|b|$ divides by zero when an operand's numerator is zero, so that a legal computation such as $(1/2) + 0$ faulted. Replacing the division test with a multiply-overflow intrinsic—a and b are both checked, the zero case short-circuits, and the overflow flag is read directly—fixes the crash and matches the no-guard performance. Replacing the division-based test, moreover, is not the only win: the C++ rational arithmetic performed one more greatest-common-divisor normalization than necessary in each of its compound operations, and removing the redundant normalization, together with a coprime fast path, cut the cost of assignment-additions by more than half. The general lesson is the one the measurements forced: profile first, and remove the *right* work.

X. Discussion and Limitations

The framework presented here is attractive precisely because its parts are so few, and the limitations are the mirror image of that economy.

Exactness stops at the rationals. The library computes exactly over the integers and the rationals and stops there. The source contains no square root, no trigonometric function, and no floating-point or complex arithmetic, and the constructions whose exact result would leave the rationals are deliberately not performed. Intersection of a line with a conic is declared but unimplemented—it returns the empty list—precisely because the general solution requires a quadratic surd; a rotation is built from a caller-supplied, already-rational cosine and sine, so a 45° rotation is representable only through a rational approximation supplied from outside; and the general projective transform, which would solve a linear system, is likewise a stub. The guarantee is therefore a *scope boundary* fixed at the field of computation, not a symbolic or floating-point fallback: no such fallback exists. An application that needs irrational or complex coordinates must extend the arithmetic itself.

Confinement to the projective plane. The framework is $\mathbb{P}^2$ and nothing higher. A coordinate is a triple, enforced by a length check; join and meet are the three-vector cross product; the conic and the transform are 3×3 matrices. In $\mathbb{P}^3$ this economy collapses. The join of two points is a line, the meet of two planes is a line, and a line is neither a point nor a plane: it is an element of the Grassmannian, represented by Plücker coordinates (a 2-blade of six components), and join and meet are realized by the exterior, or Grassmann-Cayley, products rather than by a single cross product. The one-operation duality of this paper is thus an accident of two dimensions; extending it needs an algebra of grades, not a larger cross product. The naming in the code is a reminder of the gap rather than a bridge across it: the function `plckr` and its test `test_plucker` are named after Plücker but implement the two-term combination $\lambda a + \mu b$ of the parameterization, not Plücker coordinates.

Coordinate growth. Join and meet are bilinear, so each construction doubles the degree of the coordinates in the primitive inputs; a chain of d constructions produces coordinates of degree growing by a factor of two per step, hence bit-lengths that grow by that factor in the worst case. The gcd normalization of the rational layer keeps coordinates coprime but does not bound their degree, and the fixed-width integer paths of the Rust and C++ cores can overflow well before a long construction finishes. The standard remedies—arbitrary-precision integers, modular or evaluation-interpolation arithmetic that controls the degree along the construction, and filtered predicates that fall back to exact arithmetic only when a floating-point sign is uncertain [4], [5]—are not implemented; the library takes the simplest correct path and inherits its growth. Bounding that growth over long construction chains is the clearest direction for future work.

Table III: Cross-language microbenchmarks of core operations. C++ figures are per-call times from Google Benchmark; Python figures are per-call means from pytest-benchmark. The Python column is dominated by interpreter overhead, not by arithmetic.

Operation	C++ (ns)	Python (μs)	Ratio
Dot product	2.6	1.16	440
Cross product	2.5	1.71	680
Incidence	2.5	1.50	600
Parametrize	2.5	2.79	1,110
Harmonic conjugate	2.9	13.2	4,650

Non-additivity and interoperability. Quadrance and spread are quadratic and not additive: distances add along a line and angles add about a point, but quadrances and spreads do not, so composing them requires the polynomial identities of Section VII rather than ordinary arithmetic. This is the price of exactness and it is real, especially for code ported from a trigonometric setting. It is not a disconnection from mainstream geometry, however. Distances and angles are recovered from quadrance and spread by a single square root or arcsine at the boundary, on request, and the transformations of the framework—the projective group $PGL(3)$ in the plane and the Möbius group $PGL(2)$ on the line—are exactly the Lie groups of the conventional treatment. Rational trigonometry is a measurement representation that keeps the arithmetic exact for as long as possible, not a rival to vector calculus.

Complex objects. Polarity is defined for every Cayley-Klein plane, but the self-conjugate points of the fundamental conic may be complex, and the tangents used in the angle-bisector construction have complex coordinates in elliptic geometry even though the bisectors that result are real. Supporting complex coordinates is therefore not optional for a general implementation; it is why the underlying arithmetic is over a field rather than an ordered field, and why the complex numbers appear even in constructions whose outputs are real.

Overflow and normalization. Exact integer arithmetic trades one class of error for another. Products of coordinates can overflow a fixed-width integer, and the remedy—wrapping arithmetic in Rust, arbitrary precision in Python, and a careful guard in C++—is a per-operation cost. The benchmark of Section IX-D shows that the cost is real but that a well-placed intrinsic removes most of it. For genuinely unbounded inputs, only arbitrary-precision arithmetic is safe, and its cost is the price of the guarantee.

When to use the framework. The projective-Cayley-Klein-rational chain is warranted when the geometry is not fixed in advance, when the number of special cases is large, or when the measurement must be exact. When the geometry is a single well-understood one and floating point is adequate, a special-purpose implementation will be faster and simpler. The value of the framework is uniform treatment, not raw speed.

XI. Conclusion

We have argued that projective geometry provides a single computational framework for the geometry that a curriculum usually splits apart. Beginning with homogeneous coordinates over an arbitrary field, the dot product and the cross product supply incidence, join, meet and duality; the classical theorems of Pappus, Desargues and Pascal become polynomial identities that a computer algebra system can certify; conics and polarity introduce the fundamental conic; and the Cayley-Klein construction turns that conic into a metric, with elliptic, hyperbolic and Euclidean geometry differing only in the action of the polarity. Rational trigonometry then supplies the measurements—quadrance and spread—as rational functions of the coordinates, and the five laws that govern them are the rational analogues of the sine law, the cosine law and the Pythagorean theorem. The same identities extend to polygons through the Archimedes and quadruple-quad functions, Heron's and Brahmagupta's formulas, and the rational form of Ptolemy's theorem. The projective line recovers the Mobius transformations and the velocity composition of special relativity as a special case.

The framework compiles. The `projgeom` library realizes the axiomatic hierarchy as a four-layer trait and class design in which a pure projective object is unable to compute a perpendicular, and it carries exact integer and rational arithmetic from the incidence layer up to the metric layer. Three implementations in Python, Rust and C++ agree on a shared battery of thirty reference cases and are benchmarked; the C++ and Rust cores execute a primitive operation in a few nanoseconds, and the Python core remains within interactive budgets. The cost of exactness—overflow guards and gcd normalizations—is measurable and, for in-range data, almost entirely recoverable by profiling and by replacing the right operations. The outstanding limitations are that exactness stops at the rationals, that the framework is confined to the projective plane, that coordinate bit-length grows along long construction chains, and that the rational measurements are not additive.

XII. Code Availability

The reference implementation, together with the cross-language ports and the benchmark suites, is available as `projgeom-py` (https://github.com/luk036/projgeom-py), `projgeom-rs` (https://github.com/luk036/projgeom-rs)

and `projgeom-cpp` (https://github.com/luk036/projgeom-cpp).

[1] H. S. M. Coxeter, *Projective geometry*, 2nd ed. New York: Springer-Verlag, 1974.

[2] F. Klein, *Vergleichende betrachtungen über neuere geometrische forschungen*. Erlangen: A. Deichert, 1872.

[3] N. J. Wildberger, *Divine proportions: Rational trigonometry to universal geometry*. Sydney: Wild Egg Books, 2005.

[4] C.-K. Yap, *Fundamental problems of algorithmic algebra*. New York: Oxford University Press, 2000.

[5] J. R. Shewchuk, "Adaptive precision floating-point arithmetic and fast robust geometric predicates," *Discrete & Computational Geometry*, vol. 18, no. 3, pp. 305–363, 1997.

[6] L. Kettner, K. Mehlhorn, S. Pion, S. Schirra, and C. Yap, "Classroom examples of robustness problems in geometric computations," *Computational Geometry: Theory and Applications*, vol. 40, no. 1, pp. 61–78, 2008.

[7] H. S. M. Coxeter, *Introduction to geometry*. New York: John Wiley & Sons, 1961.

[8] O. Veblen and J. W. Young, *Projective geometry*, vol. 1. Boston: Ginn; Company, 1910.

[9] P. Samuel, *Projective geometry*. in Undergraduate texts in mathematics. New York: Springer-Verlag, 1988.

[10] D. Pedoe, *Geometry: A comprehensive course*. New York: Dover Publications, 1988.

[11] R. Hartshorne, *Geometry: Euclid and beyond*. New York: Springer, 2000.

[12] J. Richter-Gebert, *Perspectives on projective geometry: A guided tour through real and complex geometry*. Berlin, Heidelberg: Springer, 2011.

[13] J. Stillwell, *The four pillars of geometry*. New York: Springer, 2005.

[14] I. M. Yaglom, *A simple non-euclidean geometry and its physical basis*. New York: Springer-Verlag, 1979.

[15] N. J. Wildberger, "Universal hyperbolic geometry I: trigonometry," *KoG*, vol. 14, pp. 24–56, 2010.

[16] W.-S. Luk, "Projgeom-py: Projective geometry in python." https://github.com/luk036/projgeom-py, 2026.

[17] W.-S. Luk, "Projgeom-rs: Projective geometry in rust." https://github.com/luk036/projgeom-rs, 2026.

[18] W.-S. Luk, "Projgeom-cpp: Projective geometry in c++." https://github.com/luk036/projgeom-cpp, 2026.

[19] W.-S. Luk, "Fractions-cpp: Header-only rational arithmetic in c++." https://github.com/luk036/fractions-cpp, 2026.

[20] W.-S. Luk, "Fractions-rs: Rational arithmetic in rust." https://github.com/luk036/fractions-rs, 2026.